

팀이 필요하다. 사람 말고.

1인 개발자를 위한 Claude Code · 에이전트 · 하네스 설계

03:17

팀이 필요하다. 사람 말고.

프로로그 · 팀이 필요하다. 사람 말고.

지난주에 세 번, 새벽에 눈을 뜨셨습니까?

그런 새벽은 대개 이렇게 시작됩니다.

새벽 3시 17분. 휴대폰이 진동합니다.

[Nightwatch] PageMark · 결제 API 응답 지연 감지 · 서킷브레이커 1차 트리거

이불 속에서 화면을 봅니다. 알림 아래로 메시지 한 줄이 따라 올라옵니다.

[backend-diagnosis] 외부 결제 파트너의 응답 지연이 원인으로 판단됩니다. 재 시도 3회 모두 실패. 결제 요청은 Fallback 큐로 전환했습니다. 사용자 영향 0건입니다.

또 한 줄.

[deploy-guardian] 최근 24시간 배포 이력을 검사했습니다. 관련 변경 없음. 현재 구성 유지를 권고합니다.

15분쯤 지난 뒤 마지막 한 줄이 올라옵니다.

[Nightwatch] 결제 API 정상 응답 복귀. 상황 종료합니다.

저는 휴대폰을 덮어둡니다. 알람은 오전 8시입니다. 그때까지는 잘 수 있습니다.

5년 전이었다면 같은 새벽을 이렇게 보냈을 겁니다. 불을 켜고, 노트북을 열고, 로그 레벨부터 조정합니다. 원인을 찾는 동안 한 시간이 지나갑니다. 팀원을 깨울 수는 없습니다. 새벽에 누군가를 깨우는 일은 하고 싶지 않으니까요. 혼자 해결하고, 해결한 뒤에야 아침을 기다립니다. 아침이 오면 다른 일이 기다리고 있습니다. 그리고 그 다음 주에도 비슷한 새벽이 옵니다.

지금은 그 일을 제가 만든 에이전트들이 합니다. 저는 혼자입니다. 하지만 혼자가 아닙니다.

이 책은 그 밤을 당신의 것으로 만드는 방법에 관한 것입니다.

이 책에 없는 것

이 책은 **어떻게**에 답하지 않습니다. **CLAUDE.md** 를 어떤 템플릿으로 쓰고, 에이전트 **.md** 를 몇 줄로 구성하고, hooks의 settings.json을 어떤 JSON으로 짜는지 — 그런 레시피는 담지 않았습니다.

이 책이 답하는 것은 **왜**입니다. 왜 프로토콜이 프롬프트를 이기는가? 왜 하네스가 모델 성능보다 먼저인가? 왜 같은 에이전트가 자기 결과를 평가하면 항상 9점이 나오는가? 왜 완벽한 운영을 목표로 하면 무너지는가?

레시피는 다른 책으로 넘겨 두었습니다. 부록에 그 책의 이름을 적어 두었습니다. 이 책을 덮고 나면, 당신은 **어떤 레시피를 쓸지가 아니라, 왜 레시피가 필요한지**를 먼저 알게 됩니다.

이 책이 전제하는 것

이 책은 두 가지를 전제합니다.

첫째, 당신은 이미 Claude Code, Cursor, Copilot 같은 AI 코딩 도구를 써 본 적이 있습니다. 어느 정도 익숙해졌습니다. 그런데 어딘가 답답합니다. 빠르게 타이핑하는 보조는 되는데, 진짜로 일을 맡길 수는 없습니다.

둘째, 당신은 혼자 혹은 아주 작은 규모로 일합니다. 사람을 뽑을 여력이 없거나, 뽑고 싶지 않거나, 뽑기 전까지 혼자 버텨야 합니다. 그러는 동안에도 서비스는 운영되어야 하고, 장애는 제때 잡혀야 하고, 새 기능은 나가야 합니다.

이 두 가지가 당신의 이야기라면, 당신은 이 책의 주인공입니다.

이 책이 따라갈 길

이 책은 다섯 걸음으로 당신을 데려갑니다. 각 걸음은 **하나의 사상**에 해당합니다.

첫 걸음에서 당신은 **적의 정체**를 보게 됩니다. 왜 AI를 매일 쓰는데도 여전히 바쁘지, 세 가지 중력이 어떻게 당신을 끌어내리는지.

두 번째 걸음에서 **프로토콜**이라는 사상을 만납니다. 프롬프트로는 도달할 수 없는, 지식을 영속화하는 방식입니다.

세 번째 걸음에서 **하네스**라는 태도를 만납니다. 좋은 모델을 기다리는 대신 좋은 환경을 설계하는 일입니다.

네 번째 걸음에서 **자기 채점의 함정**을 봅니다. 왜 같은 자리에서 만들고 평가하면 항상 합격이 나오는지.

다섯 번째 걸음에서 **스케일의 철학**을 만납니다. 완벽하게 운영하려 하지 않고 **안망하게** 운영하는 사상입니다.

그리고 책의 마지막 페이지에서 당신은 다시 프롤로그의 새벽으로 돌아갑니다. 이번에는 그것이 **당신의 새벽**입니다.

제목을 다시 봅니다.

팀이 필요하다. 그건 이미 당신이 알고 있는 사실입니다. 새벽마다 당신을 깨운 장애, 주말마다 당신을 책상 앞으로 끌어낸 운영 부담, 다음 프로젝트를 미루게 한 과부하. 그 모든 것이 **당신 혼자로는 부족하다**는 신호였습니다. 당신은 이 신호를 무시하지 않았습시다.

사람 말고. 이걸 이 책이 증명하려는 부분입니다. 사람을 새로 뽑는 일만이 팀을 만드는 길은 아니라는 것. 1인 개발자의 손끝에서 **팀의 기능을 설계로 구현**할 수 있다는 것.

설계가 가능한 이유는 생각보다 단순합니다. 당신이 이미 쓰고 있는 그 AI 도구가 **도구가 아니라 팀이 될 수 있는 구조**를 가지고 있기 때문입니다. 그 구조를 이 책에서는 다섯 개의 사상으로 풀어 봅니다.

페이지를 넘기십시오. 첫 문은 가볍습니다.

Ch 1 · 손으로 때우는 악순환의 정체 — 세 중력

0. 이 장의 문장 하나

AI 도구를 매일 쓰는데도 여전히 바쁘다면, 당신을 끌어내리고 있는 건 게으름이 아닙니다. 세 가지 **중력**입니다. 이 장은 그 적의 얼굴을 마주하는 자리입니다.

1. 먼저, 지금 당신이 어디에 서 있는지

사상을 꺼내기 전에 위치를 먼저 확인합니다. AI 코딩은 다섯 개의 층으로 나눌 수 있습니다. 각 층의 이름과 특징은 이렇습니다.

Level 5 · 에이전트 시스템	(자율 운영 + 크론 + 품질 게이트)
Level 4 · 프로토콜	(CLAUDE.md 기반 · 세션 공통 지식)
Level 3 · 인라인 에이전트 기본)	(Cursor · Copilot Chat · Claude Code)
Level 2 · 채팅	(ChatGPT 창에 코드 붙여넣기)
Level 1 · 자동완성	(Tab 키로 한 줄씩)

이 구분은 Andrej Karpathy가 2025년 여름에 제시한 **Software 3.0** 흐름 위에 1인 개발의 실무 관점을 덧씌워 재구성한 것입니다. 그는 한 문장으로 말합니다.

“LLM은 새로운 OS이고, 자연어가 새로운 프로그래밍 언어입니다.”

— A. Karpathy, **Software 3.0**, YC AI Startup School, 2025-06

OS 비유가 중요합니다. OS는 **그 위에 무언가 올리라고** 존재합니다. 대부분의 개발자는 이 OS를 깔아 두기만 하고 그 위에 **자기 것을** 올리지 않습니다. 그 상태가 Level 2~3입니다. 던닝-크루거의 정점이고, 이 책이 말하는 모든 문제의 출발점입니다.

위의 다섯 층은 **사다리가 아닙니다**. Level 5가 Level 4보다 우월한 상태도 아니고, 모두가 5까지 올라가야 할 이유도 없습니다. 이 책은 당신을 **지금 필요한 층**으로 한 계단 옮기는 데 도움을 주려는 것이지, 높은 층을 강요하는 책이 아닙니다.

그런데 왜 대부분이 Level 3에서 멈추는가 — 그 이유가 다음의 **세 중력**입니다.

2. 첫 번째 중력 · 컨텍스트 리셋

세션을 새로 열 때마다 당신은 **처음부터 다시** 설명합니다. 프로젝트의 기술 스택, 설계 원칙, 금지 사항, 문체, 운영 제약. 10분쯤 걸립니다. 그 10분이 하루 세 번 세션을 여는 사람에게는 30분이 되고, 한 주면 150분이 됩니다. 그 시간이 당신의 **실제 개발 시간**을 깎아 냅니다.

더 교묘한 문제는, 당신이 어느 순간부터 **설명을 생략하기 시작**한다는 것입니다. 매번 10분씩 쓸 여유가 없으니 짧게만 말합니다. AI는 정보가 부족하니 짐작합니다. 짐작 중 몇 개는 맞고 몇 개는 틀립니다. 틀린 것을 고치기 위해 당신은 결국 10분짜리 설명을 **수정 지시의 파편**으로 나눠서 나중에 쏟아 붓습니다. 전체 비용은 줄지 않습니다. 분산됐을 뿐입니다.

이 중력의 이름은 **컨텍스트 리셋의 중력**입니다. 세션이라는 문이 닫히면 당신이 건넌 지식도 그 방에 남아 사라지고, 새 방에서는 다시 처음부터입니다.

3. 두 번째 중력 · 품질 불안

Level 3에서는 AI의 결과물을 계속 의심해야 합니다. 의심은 합리적이지만, 의심에는 비용이 있습니다.

의심은 이런 질문으로 나타납니다. 이 코드가 우리 팀 컨벤션에 맞는가? 이 설계가 우리가 예전에 폐기한 패턴은 아닌가? 이 라이브러리 함수는 실제로 존재하는 게 맞는가? 이 변수명은 스타일 규칙에 맞는가?

네 가지를 모든 결과물마다 던지는 동안 당신의 시선이 AI 결과물 위에 얹혀 있어야 합니다. AI가 일을 하는 동안 당신의 눈은 쉬지 못합니다. 빠르게 답이 와도 빠르게 넘어가지 못합니다. 체감 속도가 Level 2 시절과 크게 다르지 않은 이유가 여기 있습니다.

더 교묘한 문제는 의심이 비일관적이라는 것입니다. 오늘은 꼼꼼하게 검토하고, 내일은 피곤해서 속 보고 통과시킵니다. 속 본 날에 실수가 프로덕션으로 새어나 갑니다. 2주 뒤에 그 실수를 수습하는 데 새벽을 씁니다. 새벽을 쓰고 나면 그 다음 주에 당신은 더 꼼꼼히 검토하려 합니다. 품질 불안은 그렇게 피로와 과잉 검토를 오가는 진자 운동으로 당신을 지치게 합니다.

이 중력의 이름은 품질 불안의 중력입니다. AI의 결과물을 완전히 믿지도, 완전히 불신하지도 못하는 상태가 고착되면, 당신의 뇌는 일이 끝난 뒤에도 쉬지 못합니다.

4. 세 번째 중력 · 스케일

혼자서 서비스 하나를 굴릴 때는 느껴지지 않는 중력이 있습니다. 서비스가 둘이 되는 순간부터 시작됩니다.

매주 점검해야 할 것이 두 배가 됩니다. 배포 게이트가 두 군데에 걸립니다. 모니터링 대시보드가 두 개로 늘어납니다. 장애 알림이 두 채널에서 옵니다. 서비스 하나 늘 때마다 새로 세팅해야 하는 문서와 에이전트와 cron 작업이 있습니다. 하지 않아도 됩니다. 다만 하지 않으면 서비스 하나가 조용히 썩어 갑니다.

1인 운영에서 가장 위험한 구조는 **수기 작업이 서비스 수에 선형 비례**하는 구조입니다. 이 구조에 한 번 들어가면, 서비스가 늘 때마다 당신의 저녁이 줄고, 주말이 줄고, 결국 **새 서비스를 시도할 여유**가 사라집니다. 더 늘릴 수 없는 상태가 되는 겁니다. 그리고 한편 기존 서비스도 관리가 부족해서 **서서히 무너지기** 시작합니다. 이중 압박입니다.

이 중력의 이름은 **스케일의 중력**입니다. 당신의 운영 패턴이 서비스 수에 선형으로 비례한다면, 당신은 언제든 이 중력에 무릎을 꿇습니다.

5. 세 중력은 서로를 먹이로 삼습니다

세 중력은 독립적으로 존재하지 않습니다. 서로를 **증폭**시킵니다.

컨텍스트 리셋이 심하면 AI 결과물의 **정합성**이 떨어집니다. 정합성이 떨어지면 **품질 불안**이 커집니다. 품질 불안이 커지면 수기 검토 시간이 늘고, 수기 시간이 늘면 **새 서비스에 투자할 여유**가 줄어듭니다. 서비스가 늘지 않으면 경험이 안 쌓이고, 경험이 안 쌓이면 다음 세션에서 AI에게 전달할 맥락의 밀도도 낮아집니다. 그래서 다시 컨텍스트 리셋이 심해집니다.

순환입니다. 그리고 이 순환은 **빨라질수록** 심해집니다. 당신이 바쁠수록 더 바빠지는 구조. 1인 개발자들이 흔히 말하는 “열심히 할수록 더 지치는” 감각의 정체가 이 순환입니다.

이 책의 나머지는 이 순환을 **한 지점씩** 깨는 이야기입니다. 한 번에 다 깨려고 하면 실패합니다. 한 군데를 깨면 나머지가 천천히 느슨해집니다. 그 첫 지점이 2장의 **프로토콜**입니다.

6. 자가진단 — 당신은 어디에 있습니까?

세 중력이 지금 당신에게 얼마나 무겁게 걸려 있는지 잠깐 확인해 봅시다. 절반 이상이 ✓ 되면 이 책은 당신을 위한 것입니다.

- ☐ Claude Code 또는 동급 도구를 **매일** 쓰고 있습니다.
- ☐ 그런데 결과물을 **매번 많이 고쳐서** 씁니다.
- ☐ 같은 맥락·같은 제약을 **주 3회 이상 다시 설명**하고 있습니다.
- ☐ 내 프로젝트에는 상수 파일(**CLAUDE.md** 또는 동급)이 **비어 있거나 없습니다**.
- ☐ 내 프로젝트에는 역할별 에이전트 파일이 **하나도 없습니다**.
- ☐ 코드 저장·커밋 시점에 자동으로 검사하는 hooks가 **설정되어 있지 않습니다**.
- ☐ 밤새 자동으로 돌아가는 점검 작업이 **하나도 없습니다**.
- ☐ 최근 한 달에 **새벽에 깨어 수습한 장애**가 한 번 이상 있었습니다.

세 중력 중 어느 것이 가장 무거운지도 짐작이 될 겁니다. 체크한 항목들의 패턴을 보면 됩니다.

1·2·3 항목이 많다면 **컨텍스트 리셋**이 주된 적입니다. 2장에서 먼저 약해집니다.

4·5·6 항목이 많다면 **품질 불안**이 주된 적입니다. 3·4장에서 약해집니다. 7·8 항목이 많다면 **스케일**이 주된 적입니다. 5장에서 정면으로 다룹니다.

7. 내일 30분에 할 것

- ☐ 6절 자가진단 8문항을 **지금** 체크하세요.
- ☐ 세 중력 중 당신에게 가장 무거운 것 한 줄을 적어 두세요.
 - 예: "컨텍스트 리셋 — 지난주 같은 맥락을 6번 다시 설명했다"
 - 예: "품질 불안 — 매 PR마다 AI 결과물 검토에 1시간"
 - 예: "스케일 — 서비스 4개, 수기 점검에 주 10시간"
- ☐ 그 한 줄을 잘 보이는 곳에 붙여 두세요. 책상 포스트잇, 메모 앱, 어디든.
- ☐ 이 책을 읽는 동안 그 한 줄이 **조금씩 약해지는지** 체감해 보세요.

이 한 줄이 책 전체의 **눈금**이 됩니다. 없으면 당신은 책을 다 읽고도 "달라진 게 있긴 한데 뭐지?" 상태에 남습니다. 한 줄짜리 눈금 하나로 그 상태를 피할 수 있습니다.

인용·참고 - A. Karpathy, **Software 3.0**, YC AI Startup School, 2025-06. - 세 중력(컨텍스트 리셋·품질 불안·스케일) 프레임은 저자가 1인 운영 경험에서 정리한 개념입니다. - Level 1~5 모델은 Karpathy의 Software 3.0 흐름 위에 1인 개발 실무 관점을 덧붙인 것입니다.

Ch 2 · 프로토콜이라는 사상 — 지식은 세션 이 아니라 파일에 산다

0. 이 장의 문장 하나

프롬프트를 **잘 쓰는** 방향으로 갈수록 당신은 지칩니다. 프로토콜을 **한 번 만드는** 방향으로 가면 당신의 타이핑이 사라집니다. 두 방향은 비용 구조 자체가 다릅니다.

1. 프롬프트와 프로토콜은 다릅니다

두 단어가 비슷하게 들립니다. 실제로는 반대 구조입니다.

프롬프트는 요청마다 다시 씁니다. 한 번 쓴 프롬프트는 세션이 닫히면 사라집니다. 요청의 수가 N 번이면 프롬프트 쓰기 비용도 N 번 발생합니다. 비용 구조가 **선형**입니다. $O(n)$ 입니다.

프로토콜은 한 번 씁니다. 한 번 쓴 프로토콜은 이후의 모든 세션이 **자동으로 로드**합니다. 요청의 수가 N 이든 1,000이든 프로토콜을 쓰는 비용은 한 번. 비용 구조가 **상수**입니다. $O(1)$ 입니다.

$O(n)$ 과 $O(1)$ 의 차이는 알고리즘 교과서에서만 중요한 게 아닙니다. 하루에 AI와 10~30번 대화하는 사람에게는 **삶의 구조**입니다. 프롬프트를 잘 쓰기 위해 노력할수록 당신은 매번 더 길고 정교한 프롬프트를 씁니다. 프로토콜을 만들면 그 노력을 **한 번만** 하고 끝납니다.

2. Karpathy가 말하는 Wiki

이 비용 차이가 최근의 LLM 연구에서도 같은 결론으로 수렴하고 있습니다.

Andrej Karpathy는 2026년 4월 한 발언에서 이런 아이디어를 꺼냈습니다.

“LLM을 통해 지식은 개별 질의마다 새로 만들어지는 것이 아니라, 지속적이고 일관되게 생성·유지됩니다.”

— A. Karpathy, LLM Wiki 개념, 2026-04

그는 이것을 LLM Wiki라고 부릅니다. 요점은 이렇습니다. 기존의 RAG 방식은 질문이 들어올 때마다 문서 더미를 뒤져 필요한 조각을 끄집어내고, 답을 돌려주고, 다음 질문을 위해서는 또 처음부터 뒤집니다. 질문이 사라지면 지식도 사라집니다. 그래서 **지식이 쌓이지 않습니다**.

LLM Wiki는 반대입니다. 문서를 **질의 시점**이 아니라 **수집 시점**에 처리해서, **영속적이고 구조화된 위키형 산출물**로 저장해 둡니다. 한 번 들어온 정보는 위키 페이지로 승격되고, 다음 관련 질의는 그 페이지를 기반으로 대답합니다. 질문이 사라져도 지식은 남습니다.

이 아이디어가 큰 시스템 이야기처럼 들리실 겁니다. 사실 그렇기도 합니다. 하지만 **1인 개발자의 컴퓨터 한 대에서도** 같은 원리가 성립합니다. 당신이 지금 하는 모든 AI 세션은 **당신이라는 도메인**에 대한 질의입니다. 세션마다 AI는 당신의 기술 스택을 다시 묻고, 금지 사항을 다시 묻고, 문체를 다시 묻습니다. 그 답을 **당신이 세션마다 다시 쓰고** 있습니다. 지식은 매 번 새로 만들어지고, 매 번 사라집니다.

그 지식을 한 번만 쓰고 영속화할 수 있다면? Karpathy의 Wiki가 1인 개발자에게 의미하는 바가 바로 이것입니다.

3. 프로토콜이 되는 세 장의 Wiki

1인 개발자의 LLM Wiki는 거창하지 않습니다. 거창한 지식 그래프 엔진이나 벡터 DB 없이, **마크다운 파일 몇 장**이면 시작됩니다. 그 파일들은 각자 다른 역할을 맡습니다.

3-1. 규칙의 Wiki — **CLAUDE.md**

첫 번째 Wiki는 당신의 프로젝트 **규칙**을 담습니다. 기술 스택, 설계 원칙, 금지 행동, 문체, 운영 제약. **당신이 매 세션마다 설명했던 그 모든 맥락**이 한 장에 모입니다.

중요한 것은 이 파일이 **자동으로 로드**된다는 점입니다. 당신이 호출하지 않아도, 프롬프트에 붙이지 않아도, 세션이 열리는 순간 AI가 먼저 읽습니다. 그 자동성이 O(1)을 완성합니다. 손이 개입해야 한다면 그건 다시 프롬프트입니다.

CLAUDE.md 는 보통 **진화**합니다. 처음에는 기술 스택 5줄로 시작하고, 과거 장애를 겪을 때마다 한 줄씩 추가됩니다. 6개월이 지나면 그 파일은 **당신이 밝은 지뢰의 지도**가 됩니다. AI는 그 지도를 가지고 당신 대신 걷습니다.

(어떤 섹션으로 구성해야 하는지, 각 섹션의 좋은 예시가 어떤지, 어떤 장애 사례를 어떻게 기록해야 하는지 — 같은 실전 구성 가이드는 이 책의 범위 밖입니다. 그 레시피는 『Claude Code로 1인 개발팀 만들기』의 3장에 있습니다.)

3-2. 기억의 Wiki — **MEMORY.md**

두 번째 Wiki는 당신의 **누적 기억**을 담습니다. 위치는 글로벌(`~/claude/MEMORY.md` 또는 동급)입니다. 이유는 단순합니다. 이것은 **특정 프로젝트**의 이야기 아니라 **당신이라는 개발자**의 이야기이기 때문입니다.

어떤 함정을 반복해서 밟았는지. 어떤 도구를 폐기하기로 결정했는지. 어떤 검토 패턴을 선호하는지. 야근한 주의 커밋이 2주 뒤 품질 이슈를 얼마나 만들었는지. 이런 메타 패턴은 프로젝트 파일에 적으면 **회발**됩니다. 프로젝트는 끝나지만 당신은 끝나지 않으니까요.

MEMORY.md의 한 줄 규칙은 이것입니다. **당신이 실제로 반복해서 겪은 것만 씁니다.** 책에서 읽은 좋은 원칙이나 가상의 교훈은 여기 섞지 않습니다. 섞이면 AI가 어느 것이 당신의 실제 경험인지 구분하지 못합니다. 기억의 **무게**가 흐려집니다.

3-3. 상태의 Wiki — **progress.md** 와 **ops/**

세 번째 Wiki는 **지금 이 프로젝트**의 상태입니다. 어제 어디까지 했는지, 지금 막힌 것이 무엇인지, 다음 세션이 먼저 봐야 할 것이 무엇인지. 이 파일이 있으면 새 세션은 **10초 안에** **어제의 자기**를 이어받습니다.

이 아이디어도 우연이 아닙니다. Anthropic이 장시간 실행 에이전트를 연구하면서 발견한 패턴이 같은 구조입니다. 그들은 이것 **structured artifacts**라고 부릅니다. feature 목록 파일, progress 파일, git 히스토리, init 스크립트. 이 네 파일만 있어도 새 세션이 컨텍스트 윈도우를 초기화해도 **상태**는 이어집니다. 에이전트가 인간 엔지니어가 아침에 출근해서 **어제 퇴근 전 메모**를 먼저 보는 구조를 모방한 것입니다.

당신의 프로젝트 규모가 커지면 **progress.md** 한 장으로는 부족합니다. 미결 이슈는 **ops/open.md** 로, 사이클 회고는 **ops/summary.md** 로 분리합니다. 파일이 많아 보이지만, 각자 **다른 것**을 담고 있으면 쓰레기통이 되지 않습니다. 경계가 흐려지면 모든 파일이 무용지물이 됩니다.

4. 디렉토리가 곧 사상입니다

세 Wiki가 맞물리기 시작하면 **디렉토리 구조 자체가 지식 체계**가 됩니다. 파일 몇 장이 흩어져 있는 것이 아니라, **서비스·규칙·기억·상태·이슈·사이클**을 각자 명시적 타입으로 굳힌 구조가 됩니다.

이 구조가 의미하는 바를 한 단어로 표현하면 **온톨로지**입니다. 도메인을 암묵지로 두지 않고 **타이핑**해 두는 일. 에이전트가 어느 파일을 언제 읽어야 하는지가 파일 이름만 보고도 유추 가능한 구조.

이 책은 온톨로지의 철학적 전통을 깊이 다루지 않습니다. 다만 이름을 알고 있으면 이후 설계가 달라집니다. "`open.md`에 열린 이슈, `summary.md`에 닫힌 사이클" 같은 구분이 **당신의 게으름**이 아니라 **의도된 지식 구조**가 됩니다.

5. 프롬프트 잘 쓰기를 포기할 때

이 사상을 받아들이고 나면 이상한 현상이 생깁니다. **당신이 프롬프트에 쏟는 시간이 오히려 줄어듭니다**. 10분짜리 맥락 설명이 0초가 되기 때문입니다. 지시는 **한 줄만 씁니다**. "비밀번호 재설정 API 만들어 주세요." 그게 끝입니다. 맥락은 이미 파일에 있습니다.

처음에는 이 짧은 지시가 어색합니다. "AI가 알아서 해 줄까?" 의심이 듭니다. 몇 번 해 보면 알게 됩니다. AI가 더 **정확히** 해 줍니다. 당신이 매번 입력하던 맥락 중 어떤 것은 빠뜨렸고, 어떤 것은 대충 넣었고, 어떤 것은 **당신이 모르고 있던 우리 프로젝트의 규칙**이었습니다. 파일은 그 모든 것을 **빠지지 않고, 일관되게** 전달합니다.

프롬프트 엔지니어링을 포기한다는 말은 **덜 노력한다**는 말이 아닙니다. 노력의 방향이 바뀌는 것입니다. 매번 쓰는 프롬프트를 **잘 쓰려** 하는 대신, 한 번만 만드는 **파일을 잘 만들려** 합니다. 시간은 여전히 들지만 **되돌아옵니다**.

6. 이 사상을 실천으로 옮기려면

이 장은 왜 **프로토콜인가**에 답하는 자리입니다. 어떻게 **CLAUDE.md**를 쓰는가는 답하지 않았습니다. 필수 섹션의 구체 목록, 4단계 진화의 실례, 글로벌과 프로젝트 계층의 분리 규칙, **MEMORY.md**의 구체 포맷, **progress.md**의 3줄 템플릿 — 이 모든 레시피가 필요하다면 『Claude Code로 1인 개발팀 만들기』의 Ch3~4에서 바로 이어 쓸 수 있습니다. 이 책의 부록에서 그 책의 챕터 매핑을 안내합니다.

이 책은 당신의 사상을 흔들려 썼습니다. 손을 움직이는 레시피는 그 뒤에 와야 효과가 납니다.

7. 내일 30분에 할 것

- ☐ 지금 가장 자주 여는 프로젝트 하나를 고릅니다.
- ☐ 빈 **CLAUDE.md**를 만듭니다. 완벽하게 쓸 필요 없습니다.
- ☐ 기술 스택 5줄 + 지금까지 AI에 반복 설명한 것 3줄 만 씁니다. 다른 섹션은 다음 장에 때 한 줄씩 추가합니다.
- ☐ 다음 세션이 그 파일을 자동으로 로드하는지 확인합니다.
- ☐ 같은 요청을 **어제 같은 방식**과 **새 CLAUDE.md** 있는 세션에서 각각 던져 봅니다. 30초만 비교합니다.

한 번만 체감하면 됩니다. 그 30초의 비교가 이 장의 사상을 당신의 몸으로 만듭니다.

인용·참고 - A. Karpathy, **LLM Wiki** 아이디어, 2026-04 공개 발언 요약
인용. - Anthropic Engineering, **Effective harnesses for long-running
agents** — structured artifacts로 세션 간 상태를 잇는 구조. - 온톨로지
철학 전통 — 본 책은 **방식을 차용**했을 뿐 전통 자체를 다루지 않습니다. -
실전 레시피 crossref: 『Claude Code로 1인 개발팀 만들기』Ch3~4.

Ch 3 · 하네스 엔지니어링이라는 태도 — 좋은 모델을 기다리지 마십시오

0. 이 장의 문장 하나

모델은 충분히 좋아졌습니다. 그런데도 당신이 AI에게 큰 일을 맡기지 못하는 이유는 **모델의 한계**가 아니라 **모델에게 주는 환경의 빈곤**입니다. 그 환경의 이름이 **하네스**입니다.

1. Anthropic이 직접 말한 문장

이 장의 주제를 Anthropic이 한 문장으로 써 두었습니다.

“Even a frontier coding model like Opus 4.5... will fall short of building a production-quality web app if given only a high-level prompt.”

— Anthropic Engineering, **Effective harnesses for long-running agents**

번역하면 이렇습니다. **Opus 4.5** 같은 최전선 모델이라도 **고수준 프롬프트 하나만 주면 프로덕션 품질의 웹앱을 만들 수 없다**.

주목할 점은 이것이 **모델을 만드는 회사가 직접 한 말**이라는 겁니다. 자기네가 파는 모델을 자기네가 “이것만으로는 안 된다”고 말했습니다. 대체 무엇이 더 필요하다는 뜻일까요. 답은 **하네스**입니다.

2. 하네스가 뭐니까

말이 이끄는 마차에 쓰는 **마구**를 하네스라고 부릅니다. 말이 아무리 힘이 세도 하네스가 없으면 마차는 어디로도 가지 않습니다. 말의 근력을 **방향**이 있는 **움직임**으로 바꿔 주는 장치가 하네스입니다.

AI 에이전트에게도 같은 구조가 필요합니다. 모델의 생성 능력은 말의 근력입니다. 그 근력을 **방향** 있는 **일**로 바꿔 주는 환경이 필요합니다. 그 환경은 다음 다섯 가지로 이루어집니다.

- **지식 계층** — `CLAUDE.md` 로 대표되는, 프로젝트의 상수 지식
- **역할 계층** — `.claude/agents/` 아래의 전문 에이전트들
- **방어 계층** — 코드가 저장·커밋되는 **그 순간**에 작동하는 hooks
- **자율 계층** — 밤새 돌아가는 cron 같은 정기 자동화
- **기억 계층** — 세션 사이를 잇는 **지속되는 산출물들**
(`MEMORY.md` · `progress.md` · feature 목록 등)

이 다섯 층 각각의 **레시피**는 이 책의 범위가 아닙니다. 이 장이 다루는 것은 **왜 다섯 층이 맞물려야 하는가**의 사상입니다.

3. 하나로는 부족한 이유

다섯 층이 왜 따로따로면 안 되는지는 실패 사례를 통해 가장 잘 드러납니다.

지식 계층만 있고 방어 계층이 없으면, `CLAUDE.md`에 "프로덕션 DB 직접 접근 금지"라고 써 두어도 피곤한 새벽의 당신은 AI의 제안을 통과시킵니다. 문서는 **권고**에 머무릅니다.

방어 계층만 있고 지식 계층이 없으면, hooks가 금지된 행동을 차단해도 에이전트가 왜 그 행동이 금지됐는지 이해하지 못합니다. 차단된 요청을 돌려서 다시 시도합니다.

역할 계층만 있고 기억 계층이 없으면, 에이전트는 어제 한 일을 기억하지 못한 채 오늘 새로운 제안을 합니다. 같은 회의를 매일 하는 셈입니다.

기억 계층만 있고 자율 계층이 없으면, 당신이 자리를 비웠을 때 아무 일도 일어나지 않습니다. 기억은 쌓이는데 그걸 읽고 행동할 주체가 없습니다.

다섯 층이 맞물릴 때 비로소 당신의 AI는 말에서 마차로 바뀝니다. 방향과 속도와 안전과 기억을 모두 가진 마차로.

4. Anthropic이 드는 두 에이전트 구조

Anthropic의 Effective harnesses 연구는 이 하네스 사상의 가장 단순한 실전 형태를 제시합니다. 그들은 장시간 실행 에이전트를 두 역할로 쪼갰습니다.

- **Initializer agent** — 첫 실행 시 프로젝트 뼈대와 기준 파일들을 세팅합니다.
- **Coding agent** — 이후 세션에서 실제 기능을 증분 구현합니다.

두 에이전트는 같은 인프라를 공유하지만 다른 초기 프롬프트를 받습니다. 각자의 역할이 엄격히 다릅니다. Initializer는 코드를 짜지 않고, Coding agent는 뼈대를 다시 만들지 않습니다.

이 분리가 얼마나 단순한지 보십시오. 새로운 알고리즘, 새로운 모델, 새로운 벡터 DB가 필요하지 않습니다. 역할을 둘로 쪼개는 것만으로 장시간 자동 실행의 방향성이 유지됩니다. 하네스의 힘이 드러나는 지점이 바로 이곳입니다. 구조가 행동을 결정합니다.

5. 에이전트는 매뉴얼이지 프롬프트가 아닙니다

역할 계층을 조금 더 파 봅시다. 에이전트를 잘 만드는 핵심 원리가 사상적으로 어디에 있을까요?

프롬프트는 **구두 지시**입니다. 오늘 한 말과 내일 한 말이 달라집니다. 팀장이 자리를 비우면 일이 멈춥니다. 새 사람이 들어올 때마다 같은 온보딩을 반복합니다. 사람 팀이든 AI든 이 구조는 오래가지 못합니다.

잘 돌아가는 팀에는 매뉴얼이 있습니다. **코드 리뷰 기준은 이 문서에 있어요. 배포 절차는 여기에 적혀 있어요. 이 기능은 저 패턴으로 구현해요.** 팀원 누구든 같은 기준으로 일하고, 신규 입사자도 문서만 읽으면 시작할 수 있습니다.

AI 에이전트는 정확히 이 구조입니다. `.claude/agents/` 아래의 `.md` 파일 한 장이 에이전트 한 명입니다. YAML frontmatter로 식별되고, 역할·매뉴얼·금지 목록으로 행동이 정해집니다. **프롬프트를 치는 건 구두 지시, 에이전트를 만드는 건 업무 매뉴얼을 만드는 것입니다.** 이 두 문장은 같은 일의 물리적 결과가 다른 이유를 설명합니다.

그 연장선에서 SRP(Single Responsibility Principle) 하나만 기억하십시오. 에이전트에도 단일 책임이 필요합니다. **만능 에이전트 1개**보다 **전문 에이전트 5개**가 낫습니다. 만능은 컨텍스트가 흐려지고, 자기 채점을 시작하고, 유지보수가 무서워집니다. 다섯으로 쪼개면 각자 100줄짜리 매뉴얼이 됩니다. 한 명을 고쳐도 다른 넷이 멀쩡합니다. 이 원리는 이미 60년 된 소프트웨어 원리의 재적용입니다. 새로울 게 없습니다. 새로운 건 **AI에게도 같은 원리가 적용된다**는 관찰입니다.

6. 컨텍스트 엔지니어링 — 정보를 통과시키는 법

하네스의 기억 계층을 더 정교하게 다루려면 한 개념이 더 필요합니다. 같은 Karpathy가 최근에 **컨텍스트 엔지니어링**이라는 이름으로 정리한 기술입니다. 그는 다섯 개의 동사로 요약합니다.

- **선택** (select) — 전체 정보 중 **지금 이 단계에서 필요한 조각**만 고르기
- **압축** (compress) — 길이를 줄이되 의미를 잃지 않기
- **순서** (order) — 중요한 것을 앞에, 덜 중요한 것을 뒤에
- **격리** (isolate) — 서로 다른 작업의 맥락이 섞이지 않도록 분리
- **포매팅** (format) — 같은 정보라도 AI가 읽기 쉬운 형태로

큰 시스템의 용어로 들리지만, 1인 개발자의 맥락에서는 아주 구체적인 설계 원칙이 됩니다. 하네스의 역할 계층(에이전트 분리)이 바로 **격리**의 실전입니다. 메인 세션이 서브 에이전트에게 일을 넘기면 **새 컨텍스트**가 열립니다. 메인의 긴 대화 기록이 서브에 묻어오지 않습니다. 서브는 깨끗한 맥락에서 자기 매뉴얼만 들고 일합니다. 결과는 파일로 돌아옵니다. 메인은 파일을 읽고 다음을 결정합니다.

이 구조는 **메인은 판단하고, 서브는 일한다**는 한 문장으로 요약됩니다. 두 역할을 같은 세션이 겸하려는 순간 컨텍스트 희석이 시작됩니다. 5절의 SRP가 여기서도 작동합니다.

7. 책임의 경계 — 위임의 사상

하네스의 방어 계층과 자율 계층이 맞물리면 **사람이 개입할 지점**을 정하는 문제가 뒤따릅니다. 모든 작업을 사람이 확인할 수는 없습니다. 반대로 모든 작업을 자동화하면 어느 순간 되돌릴 수 없는 사고가 납니다. 그 경계를 **미리** 정해 두어야 원칙이 흔들리지 않습니다.

경계는 네 층입니다.

- **L1 자동 실행** — 포맷터·린트·자동 수정. 사람이 결과를 보지 않아도 된다.
- **L2 자동 실행 + 보고** — 단위 테스트 자동 수정·타입 오류 수정. 사람은 완료 로그만 본다.
- **L3 계획 + 사람 승인** — DB 스키마 변경·의존성 추가. AI가 계획서를 내고 사람이 승인한 뒤 실행한다.
- **L4 사람이 실행** — 프로덕션 배포·결제 로직·데이터 삭제. AI는 체크리스트 까지만. 실행 트리거는 사람의 손.

이 네 층을 문서로 명문화하는 순간 매 결정이 쉬워집니다. "이건 어느 레벨이지?"라는 질문이 "이건 되돌릴 수 있나?", "실패 시 사용자 영향이 가나?"로 분해됩니다. 되돌릴 수 없으면 L4입니다. 사용자 영향이 가면 L3 이상입니다. 반복되고 영향이 없으면 L1 또는 L2로 자동화합니다.

이 분류가 hooks의 기술적 차단과 맞물릴 때 비로소 **원칙이 지켜집니다**. 문서와 경고 주석과 hooks, 세 층이 같은 원칙을 반복해서 확인합니다. 한 층이 뚫려도 다음 층이 받습니다. 다층 방어는 사상은 간단합니다. **사람의 의지에 의존하지 않는 구조**.

8. 왜 모델보다 하네스인가

이 장의 제목을 한 번 더 풀어 봅니다. **좋은 모델을 기다리지 마십시오**.

이유 네 가지입니다.

첫째, **모델은 곧 평준화됩니다**. 오늘 최상위 모델과 한 단계 아래 모델의 차이는 몇 개월짜리입니다. 최상위 모델에만 의존하면 그 차이가 사라지는 순간 강점도 사라집니다. 좋은 하네스는 **어떤 모델을 끼워도** 일정 수준 이상의 결과를 냅니다.

둘째, **하네스는 누적됩니다**. 모델은 교체되면 맥락이 리셋됩니다. 하지만 당신이 지난 3개월 쌓은 **CLAUDE.md**, 에이전트 파일, hooks 설정, cron은 교체되지 않습니다. GPT에서 Claude로 바뀌어도 하네스는 거의 그대로 이식됩니다. 당신이 쌓은 시간이 **모델이 아니라 하네스에** 남습니다.

셋째, **하네스가 실수를 막습니다**. 모델이 99% 정확해도 **나머지 1%**가 치명적일 수 있습니다. 하네스의 hooks 계층은 그 1%를 현장에서 물리적으로 차단합니다. 모델 자체로는 할 수 없는 일입니다.

넷째, **1인에게 가장 큰 레버리지입니다**. 팀이 있는 조직에서는 동료·QA·DevOps가 하네스의 일부를 대신합니다. 1인에게는 그 역할이 없습니다. 하네스 없는 1인 운영은 **시간의 문제**입니다. 언젠가는 무너집니다. 하네스가 있는 1인 운영은 **시간의 친구**입니다. 오래갈수록 단단해집니다.

9. 이 태도를 실천으로 옮기려면

이 장은 **하네스가 왜 필요한가**의 사상을 다뤘습니다. hooks의 settings.json 실제 설정, cron의 구체 엔트리, 에이전트 **.md**의 샘플 5종, pre-commit 혹은 셸스크립트, CI의 에이전트 기반 리뷰 단계 — 이 모든 실천은 『Claude Code로 1인 개발팀 만들기』의 Ch4·Ch5·Ch6에 있습니다. 이 책이 당신의 **태도**를 만들면, 그 책이 당신의 **손**을 움직입니다.

10. 내일 30분에 할 것

- ☐ 현재 프로젝트 루트에 **progress.md** 한 파일을 만듭니다.
- ☐ 그 안에 세 줄만 씁니다.
 - **##** 지금 하고 있는 것
 - **##** 지금 막힌 것

○ ## 다음 세션이 먼저 봐야 할 것

- ☐ CLAUDE.md 의 맨 위에 한 줄을 추가합니다. "새 세션 시작 시 progress.md 를 먼저 읽어 주세요."
- ☐ 오늘 세션을 마치기 전에 progress.md 를 업데이트합니다.
- ☐ 내일 아침 새 세션을 열었을 때 AI가 어제의 맥락을 얼마나 빠르게 이어받는 지 체감합니다.

이 한 개 파일이 추가되면서 당신의 프로젝트는 5계층 중 기억 계층을 처음으로 갖게 됩니다. 가장 작게 시작해서 가장 크게 체감하는 30분입니다.

인용·참고 - Anthropic Engineering, *Effective harnesses for long-running agents*, 2026. - A. Karpathy, *Context Engineering* 프레임, 2026 — 선택·압축·순서·격리·포매팅 5요소. - SRP — Robert C. Martin, SOLID 원칙 중 Single Responsibility Principle. - 위임 L1~L4 프레임은 저자가 1인 운영 경험에서 정리한 개념입니다. - 실전 레시피 crossref: 『Claude Code로 1인 개발팀 만들기』Ch4(에이전트 아키텍처)·Ch5(자동화 인프라)·Ch6(품질 게이트).

Ch 4 · 자기 채점의 함정 — Gen/Val 분리 의 인식론

0. 이 장의 문장 하나

생성과 평가가 같은 몸에 있으면, 평가는 **자화자찬**이 됩니다. 이것은 모델의 정직성 문제가 아니라 **컨텍스트의 관성** 문제입니다.

1. 모두가 한 번쯤 속는 실험

당신이 AI에게 글 한 편을 쓰게 합니다. 초안이 돌아옵니다. 나쁘지 않은 것 같습니다. 곧바로 당신은 같은 세션에 묻습니다. "이 글을 10점 만점으로 채점해 줘."

AI가 답합니다. "8.5점입니다. 구조가 명확하고 근거가 충분하며 문장이 간결합니다."

같은 일을 **다른** 세션에 가서 시켜 봅니다. 앞선 세션의 초안만 복사해 가고, 이번에는 "이 글을 10점 만점으로 채점해 줘"만 묻습니다. 점수가 돌아옵니다. 5.8점입니다. "도입이 배경 설명으로 시작하고, 근거 없는 주장이 세 문단에 걸쳐 있으며, 결론이 자기 홍보처럼 읽힙니다."

같은 초안, 같은 모델, 같은 프롬프트. 다른 점은 **누가 평가하느냐**만. 앞 세션은 자기가 만든 것을 평가했고, 뒤 세션은 **남이 만든 것**을 평가했습니다.

이 현상을 한 번이라도 직접 본 사람은 **자기 채점**이라는 단어를 절대 가볍게 쓰지 않게 됩니다.

2. 이것은 **시**만의 문제가 아닙니다

놀랍게도 같은 현상이 사람에게도 작동합니다. 방금 자기가 쓴 글을 10분 뒤에 읽으면 괜찮아 보입니다. 같은 글을 일주일 뒤에 읽으면 **낯선 사람의 글**처럼 보이고 문제점이 눈에 들어옵니다. 시간이 컨텍스트를 **분리**해 준 겁니다.

AI에게는 시간이 없습니다. 대신 **세션의 경계**가 같은 역할을 합니다. 세션이 분리되면 방금 만든 판단의 흐름에서 **떨어집니다**. 그 떨어짐이 엄정한 평가를 가능하게 합니다.

반대로 같은 세션에서는 방금 자기가 판단한 흐름을 **부정하기가 구조적으로 어렵습니다**. 자기 모순을 피하려 합니다. 합리화를 찾습니다. 그래서 9점을 줍니다. 이 건 악의가 아니라 **컨텍스트의 관성**입니다. 사람 뇌에도 같은 관성이 있습니다. **확증 편향**이라고 불리는 것이 이 관성의 한 이름입니다.

이 장의 중심 주장은 여기 있습니다. **자기 채점의 함정은 LLM의 결함이 아니다. 인지의 일반적 성질이다**. AI는 그 성질을 증폭시킬 뿐입니다. 그 증폭을 막는 방법이 **Gen/Val 분리**입니다.

3. Anthropic이 발견한 **GAN** 같은 구조

이 아이디어에 Anthropic이 실험적 뒷받침을 더했습니다. 그들은 에이전트 시스템을 **GAN 스타일**로 엮어 봤습니다.

GAN(Generative Adversarial Network)은 원래 이미지 생성 모델의 훈련 기법입니다. 생성자와 판별자가 **서로 다른 목적**을 가지고 겨루면서 품질이 올라가는 구조입니다. Anthropic의 제안은 코딩·글쓰기·에이전트 작업에도 같은 분리가 유효하다는 것이었습니다. 결론의 핵심은 이 한 문장입니다.

생성자와 평가자는 **역할**이 다릅니다. 평가자는 **주관적 판단을 구체적이고 채점 가능한 항목**으로 변환해야 합니다.

두 번째 절이 특히 중요합니다. "**더 자연스럽게**" 같은 추상 수식어로는 평가자가 쓸모없어집니다. 평가자는 grep·길이·존재 여부로 환원되는 기준이 있어야 합니다. "주장 문단의 80% 이상에 직인용 또는 수치 또는 구체 사례가 붙는다"는 기준은 측정 가능합니다. "더 좋은 근거가 있어야 한다"는 기준은 측정 불가능합니다. 후자는 자기 채점을 정당화하는 고무줄이 됩니다.

4. 4단의 사상적 뼈대

이 사상을 실무에 옮기는 구조는 **네 단계**입니다.

- **Step 1 · 생성 (Gen)** — 초안 A를 만든다. 자기 검열은 최소한으로.
- **Step 2 · 비평 (Critique)** — 다른 세션이 초안 A만 보고 약점 세 개를 뽑는다.
- **Step 3 · 평가 (Eval)** — 또 다른 세션이 미리 정해진 **루브릭**으로 점수를 매긴다.
- **Step 4 · 퇴고 (Refine)** — 비평과 평가를 근거로 초안 B를 만든다. 무엇이 왜 바뀌었는지 **트레이스**를 남긴다.

이 네 단계가 각자 **다른 세션**에서 돌아가야 의미가 있습니다. 같은 세션이 Step 1과 Step 3을 겸하면 Step 3은 Step 1의 자화자찬이 됩니다. 각자 다른 **에이전트 파일**을 쓰면 더 강하게 분리됩니다. 핵심은 **역할의 물리적 분리**입니다.

그리고 반복에는 반드시 **종료 조건**이 있어야 합니다. 두 번 연속 점수 변화가 1점 이하면 종료. 총 반복 3회를 넘지 않음. 이 종료 조건이 없으면 당신은 **영원히 퇴고**할 수 있습니다. 영원은 좋은 게 아닙니다.

5. 루브릭은 미리 만듭니다

위에서 한 번 더 강조해야 할 말이 있습니다. 루브릭을 평가자가 만들게 하지 마십시오. 평가자에게 "좋은 평가 기준을 만들고 그 기준으로 채점해 줘"를 시키면 평가자는 자기에게 유리한 기준을 만듭니다. 이 구조에서는 항상 고득점이 나옵니다.

루브릭은 파이프라인을 시작하기 전에 당신이 직접 만들어야 합니다. 4~5개 축, 각 축이 서로 독립, 점수가 검증 가능한 행동으로 서술. 평가자 에이전트에게는 "이 루브릭만 써라"를 못 박아야 합니다. 이 원칙 하나로 평가의 날이 쉽습니다.

글쓰기를 예로 들면 축은 이런 식입니다. 구조(주장-근거-예시-결론이 분리되는가), 근거(주장 문단의 몇 %에 직인용·수치·사례가 붙는가), 문장(평균 길이와 문단 당 문장 수), 고유성(저자 특유의 조어·비유·관점이 등장하는가). 네 축 모두 측정 가능합니다.

코드 작업이면 축은 이렇게 바뀝니다. 컨벤션(CLAUDE.md 위반 건수), 테스트(변경에 대응하는 테스트·엣지 케이스 커버), 방어(외부 의존의 timeout·재시도·서킷브레이커), 가독성(함수 길이·이름 정확도·주석 성격).

중요한 것은 무엇을 재는가가 아니라 구체적으로 잴 수 있는가입니다.

6. 왜 이 사상이 당신의 일 전체에 적용되는가

Gen/Val 분리는 AI 시스템에서 태어난 개념이지만, 당신의 1인 운영 전반에 같은 원리가 작동합니다.

자기 PR을 자기가 리뷰하면 맹점이 남습니다. 그래서 당신은 서브에이전트 `code-reviewer` 를 별도로 둡니다. 자기가 짠 설계 문서를 자기가 검수하면 합격이 나옵니다. 그래서 당신은 `senior-editor` 를 별도로 둡니다. 자기가 쓴 실패 일지를 자기가 판정하면 관대해집니다. 그래서 당신은 그 일지를 반드시 다음 사이클의 당신에게 읽히도록 파일로 남깁니다.

이 모두가 같은 사상입니다. 만드는 자와 평가하는 자를 물리적으로 분리해야 평가의 날이 쉽니다. 시간으로 분리하든(시간차 독해), 세션으로 분리하든(새 컨텍스트), 사람으로 분리하든(동료 리뷰), 에이전트 파일로 분리하든 — 분리의 형태만 다를 뿐 원리는 같습니다.

7. 한 걸음 더 — 의심의 관성

이 장을 닫기 전에 한 가지만 더 짚습니다. Gen/Val 분리를 받아들이는 사람이 자주 빠지는 역방향의 함정이 있습니다. 모든 일에 평가 단계를 끼워 넣으려는 충동입니다. 커밋 메시지 하나에도 평가자를 돌리고, 문서 한 줄 고치는 데도 4단 파일라인을 돌립니다.

이것은 의심의 관성입니다. 자기 채점을 피하려다가 자기 과잉 검증에 빠집니다. 결과는 같은 방향입니다. 당신의 시간이 사라집니다.

경계선은 단순합니다. 실패의 비용이 높은 산출물에만 Gen/Val 분리를 풀가동합니다. 프로덕션 영향이 가는 코드, 외부 공개 글, 계약이 걸린 문서. 그 외에는 Step 1과 Step 4만 돌립니다. Step 2·3은 중요한 순간을 위한 것입니다. 모든 것을 위한 것이 아닙니다.

8. 이 사상을 실전으로 옮기려면

이 장은 왜 자기 채점이 함정인가의 사상을 다뤘습니다. 책임 분리·자동 검증·배포 게이트를 시스템 차원에서 어떻게 짜는지는 『Claude Code로 1인 개발팀 만들기』의 Ch6(품질 게이트)에서 이어집니다. 단 Gen/Val 파이프라인의 실제 프롬프트·루브릭·4단 실행 로그는 본 책이 제시한 원리 위에 여러분이 직접 설계해야 할 영역입니다. 원리는 이 장에, 주변 게이트 구조는 워크북에, 구체 프롬프트는 당신의 손에.

9. 내일 30분에 할 것

- ☐ 이번 주에 쓸 짧은 글 또는 코드 변경 한 덩어리를 정합니다.
- ☐ 초안을 한 세션에서 만듭니다. (Gen)
- ☐ 새 세션을 열고 그 초안만 붙여넣은 뒤 "이 글(코드)의 약점 세 가지를 뽑아주세요"라고만 지시합니다. (Critique)
- ☐ 세 약점 중 적어도 하나를 실제로 반영해 2차 초안을 만듭니다.
- ☐ 1차와 2차의 차이를 30초만 비교합니다. 어느 지점이 바뀌었는지.

이 한 번의 비교가 Gen/Val 분리의 사상을 당신의 몸으로 만듭니다. 익숙해진 뒤에 평가자(Step 3)와 반복 루프까지 붙이면 됩니다.

인용·참고 - Anthropic Engineering, GAN-style 하네스·생성자·평가자 분리 관련 포스트. <https://www.anthropic.com/engineering> - 자기 채점·확증 편향의 인지 심리적 배경 — 본 책은 이 전통을 깊이 다루지 않습니다. 핵심만 차용했습니다. - 실전 레시피 crossref: 『Claude Code로 1인 개발팀 만들기』Ch6(품질 게이트).

Ch 5 · 스케일의 철학 — 완벽이 아니라 안 망하게

0. 이 장의 문장 하나

서비스가 늘어나도 당신의 시간이 같은 속도로 늘지 않게 만드는 비결은, **모든 걸 잘 굴리려 하지 않는 것**입니다. 무너지지 않으면 됩니다. 그 한 자리의 양보가 스케일을 가능하게 합니다.

1. "혼자 10개"가 정말 가능한가

1인으로 서비스 하나를 운영하는 건 지금의 도구로 충분히 가능합니다. 두 개도 가능합니다. 다섯 개부터 질문이 시작됩니다. 열 개부터는 **운영의 정의 자체**를 다시 세워야 합니다.

이 책을 여기까지 읽은 독자라면 답을 짐작할 겁니다. 가능합니다. 단, 전제가 있습니다. **모든 서비스를 같은 밀도로 굴리지 않는다는** 전제. 이 책의 1~4장을 소화한 1인 개발자라도 열 개 서비스를 모두 **메인 제품 수준**으로 관리하는 건 불가능합니다. 불필요하기도 합니다. 어떤 서비스는 메인이 맞지만, 어떤 서비스는 사이드가 맞습니다. 이 분별이 스케일의 핵심 기술입니다.

이 장은 한 질문에 답합니다. **서비스가 늘어나도 당신이 늘어나지 않는 구조는 어떻게 만드는가?**

2. 첫 번째 양보 — 완벽을 포기합니다

스케일을 막는 첫 번째 장애물은 기술이 아닙니다. **태도**입니다.

1인 개발자가 서비스를 늘릴 때 처음 부딪히는 문제는 "내가 만든 것은 모두 소중한 것"이라는 마음입니다. 그 마음은 아름답지만 운영에는 독입니다. 모든 서비스가 같은 우선순위가면, 결국 아무 서비스도 우선순위가 없는 상태가 됩니다. 당신의 저녁이 사라지고, 주말이 사라지고, 새 서비스를 시작할 에너지가 사라집니다.

태도를 바꿔야 합니다. **이 책의 목표는 완벽이 아니라 안 망하게**입니다. 이 문장은 책 전체를 관통하는 중심 명제 중 하나입니다. 다시 한 번 적어 둡니다.

완벽은 목표가 아닙니다. 안 망하는 것이 목표입니다.

이 두 문장 사이의 거리가 스케일이 가능한가 불가능한가의 경계선입니다. 완벽을 목표로 잡으면 서비스 하나마다 **무한한** 노력이 들어갑니다. 안 망하는 것을 목표로 잡으면 서비스마다 **필요한 만큼만** 들어갑니다. 후자만이 여러 서비스를 견뎌냅니다.

3. 두 번째 양보 — 티어를 받아들입니다

안 망하게만 하려면 **어떤 서비스는 덜 보살핀다**는 결정을 해야 합니다. 방치가 아니라 **의식적 집중**입니다.

서비스를 세 개의 티어로 나눕니다.

- **Tier 1 · 수익·신뢰 직결** — 장애 시 즉시 매출 손실 또는 사용자 신뢰 급락. 허용 장애 30분 이내. 분 단위 모니터링. **CLAUDE.md** 와 에이전트 전부 상주.
- **Tier 2 · 운영·부가 매출** — 장애가 나도 **당일 복구**면 사용자에게 큰 피해 없음. 허용 장애 24시간. 일 단위 점검. 에이전트는 필수 역할만.

- **Tier 3 · 실험·사이드·내부용** — 무너져도 **한 주 안에** 복구하면 됨. 허용 장애 1주일, 주 1회 확인. 에이전트는 필요할 때만.

이 분류가 **의식적**이라는 게 핵심입니다. 서비스를 Tier 3으로 **의식적으로 내린다**는 것은 "나는 이 서비스에 쏟는 시간을 줄이기로 했다"는 선언입니다. 기술적 결정이 아니라 **시간 배분의 결정**입니다.

티어 분류가 없는 1인 개발자는 자기도 모르게 **모든 서비스를 Tier 1**로 굴립니다. 결과는 예측 가능합니다. 어느 서비스도 Tier 1 수준에 도달하지 못한 채 당신의 시간만 갈려 나갑니다. 티어를 받아들이지 않으면 스케일은 없습니다.

4. 세 번째 양보 — 실패를 남깁니다

세 번째 양보는 어쩌면 가장 어렵습니다. **자신의 실패를 기록으로 남기는** 일입니다.

장애가 터지면 당신은 일단 수습합니다. 수습이 끝나면 피곤합니다. 자고 싶습니다. 그 순간에 파일 하나를 열어 **방금 일어난 일을 다섯 줄로 적는 것** — 이것이 이 장의 세 번째 제안입니다.

어떤 증상이었는지. 원인이 무엇이었는지. 즉시 어떤 조치를 했는지. 근본 조치는 무엇이었는지. 재발 방지를 위해 `CLAUDE.md` 어디에 한 줄을 추가할 것인지.

다섯 줄입니다. 5분이면 됩니다. 그 5분을 자는 시간으로 돌리면, 내일 아침 당신은 그 장애의 80%를 잊습니다. 일주일이면 95%를 잊습니다. 한 달이면 같은 장애를 **거의 같은 방식으로** 다시 겪습니다.

같은 장애를 두 번째 겪는 1인 개발자는 이 사실에 분노하지 않습니다. 익숙해져서 그렇습니다. 하지만 이 익숙함이 **스케일의 적**입니다. 실패 일지 (`FAILURES.md` 혹은 `ops/failures.md`)는 **3년 뒤의 당신을 구합니다**. 당신의 시도 그 파일을 읽고 다음 코드에 과거의 교훈을 반영합니다.

단 한 줄의 규칙이 이 일을 가능하게 합니다. **수습이 끝난 그 5분 안에 쓴다.** 다음 날은 이미 늦습니다.

5. 자동화의 두 유혹

스케일의 철학이 자리 잡으면 곧 두 가지 유혹이 찾아옵니다.

첫 번째는 **모든 서비스를 Tier 1로 올리려는 유혹**입니다. "내가 만든 건 소중하니까"라는 마음의 귀환입니다. 결과는 이미 3절에서 본 그대로입니다. 당신의 개발 시간이 사라집니다. 티어를 의식적으로 내려놓는 일을 계속 해야 합니다.

두 번째는 **모든 일을 자동화하려는 유혹**입니다. "사람 개입 없이 돌아가면 얼마나 좋을까"라는 생각입니다. 이 유혹은 종종 3장의 L4 원칙을 깨뜨리려는 쪽으로 밀어붙입니다. 새벽 3시에 자동 배포되는 결제 로직이 한 번 큰 사고를 내면, 그때 깨닫습니다. 자동화의 비용이 **가끔 나는 큰 사고**라는 것을. 그 한 번의 사고가 쌓아 둔 모든 시간 이득을 넘어서는 순간이 옵니다. L4는 유혹에 굴복하지 않는 마지막 방어선입니다.

두 유혹 모두 같은 뿌리에서 옵니다. **완전함을 바라는 마음.** 완벽한 운영, 완전한 자동화. 이 책이 처음부터 말한 한 문장이 여기서 다시 돌아옵니다. **완벽이 아니라 안 망하게.** 유혹에 흔들릴 때마다 이 문장을 꺼내십시오.

6. "사람 없이도 돌아간다"의 진짜 의미

제목 "팀이 필요하다. 사람 말고."의 두 문장이 이 장에서 가장 선명해집니다.

팀이 필요하다는 당신이 이미 알고 있던 사실입니다. 새벽 장애와 주말 수습과 미뤄진 새 프로젝트가 그 신호였습니다.

사람 말고는 이 책이 다섯 장에 걸쳐 증명하려 한 부분입니다. 증명의 방식은 간단했습니다. **설계를 통해 팀의 기능을 옮긴다**. 프로토콜로 지식을 옮기고, 에이전트로 역할을 옮기고, 하네스로 책임의 경계를 옮기고, Gen/Val 분리로 검증을 옮기고, 티어링으로 집종의 순서를 옮깁니다. 사람이 했을 일들을 하나씩 설계로 대체하면서, 결과적으로 **사람 없이 팀**이 되는 겁니다.

이 구조가 완성된 사람의 하루는 이런 모습입니다. 아침에 AI가 **open.md** 를 읽고 오늘의 작업을 L1~L4로 분류해서 보고합니다. 오후에 L3 계획 몇 건을 승인하면 AI가 실행합니다. 저녁에 L4 배포 하나를 사람의 손으로 누릅니다. 새벽에 장애가 터지면 서버에이전트가 자동 롤백하고 보고서를 남깁니다. 당신은 잡니다. 다음 날 **summary.md** 에 한 줄이 추가되어 있습니다.

이 하루는 마법이 아닙니다. 이 책의 1~5장에서 쌓은 다섯 층의 사상이 맞물려 만든 결과입니다. 한 층 한 층은 쉬웠지만, 맞물리면 커집니다.

7. 그리고 가장 마지막 양보 — 이 책을 닫는 것

스케일의 철학에 도달한 사람에게 남는 마지막 양보가 있습니다. **이 책을 닫고 당신의 파일을 여는 일**입니다.

이 책은 다섯 사상을 심었습니다. 하지만 사상은 **손이 움직이지 않으면** 아무것도 아닙니다. 내일 30분 안에 당신의 **CLAUDE.md** 첫 줄을 쓰지 않으면 이 책은 읽은 책으로만 남습니다. 읽은 책은 당신의 새벽을 구해 주지 않습니다.

다행히 손을 움직이는 레시피는 이 책 바깥에 있습니다. 부록에서 안내하는 **실전 워크북** 한 권이 당신의 다음 30일을 함께합니다. 이 책이 당신의 태도를 만들었다면, 그 책이 당신의 구조를 세웁니다.

8. 내일 30분에 할 것

- ☐ 현재 운영 중인 서비스 목록을 적습니다.
 - ☐ 각 서비스 옆에 Tier 1 / 2 / 3을 적습니다.
 - **다운타임 30분**이 매출을 의미 있게 깎으면 Tier 1
 - **하루 다운타임**이 외부 사용자에게 알림이 가면 Tier 2
 - **일주일 꺼져 있어도** 외부에서 체감하는 사람이 없으면 Tier 3
 - ☐ 가장 덜 중요한 서비스 **하나**를 고르고, 모니터링 주기를 **한 단계 낮춥니다**.
이건 실천이자 선언입니다.
 - ☐ 이번 주에 생긴 장애 1건(없으면 최근 1건)을 **FAILURES.md**에 다섯 줄로 기록해 둡니다.
 - ☐ 내일 아침 그 파일을 열었을 때, **과거의 당신이 남긴 메모를 지금의 당신이 읽는 감각**을 체감합니다.
-

9. 이 철학을 실전으로 옮기려면

티어별 **CLAUDE.md**의 구체 분량, 모니터링·크론 주기의 실제 설정, 실패 일지 템플릿의 정확한 필드, 티어 간 전환의 실무 체크리스트, 30일·90일·180일 실전 로드맵 — 이 모든 레시피는 『Claude Code로 1인 개발팀 만들기』의 Ch7(30서비스 운영)·Ch8(실패 일지)·Ch9(내일 적용)에서 바로 이어집니다.

이 장이 당신의 철학을 세웠습니다. 그 책이 당신의 운영을 세웁니다.

인용·참고 - 서비스 티어링·실패 일지·자동화 유혹 프레임은 저자의 1인 운영 경험에서 정리한 개념입니다. - Anthropic Engineering, 체크리스트 기반 게이트·장기 안정성 관련 포스트 참고. <https://www.anthropic.com/engineering> - 실전 레시피 crossref: 『Claude Code로 1인 개발팀 만들기』Ch7(30서비스 운영)·Ch8(실패 일지)·Ch9(내일 적용).

에필로그 · 당신은 더 이상 혼자가 아닙니다

새벽 3시 17분. 당신의 휴대폰이 진동합니다.

[알림] 결제 API 응답 지연 감지 · 서킷브레이커 1차 트리거

당신은 이불 속에서 화면을 봅니다. 알림 아래로 **당신이 직접 만든 backend-diagnosis** 에이전트의 보고가 이어집니다. 15분 뒤, "상황 종료"가 뜹니다. 당신은 휴대폰을 얹어두고 다시 잠듭니다.

5년 전이었다면 당신은 지금부터 아침까지 깨어 있었을 겁니다. 지금은 **당신이 직접 쓴 CLAUDE.md**와 **당신이 직접 뽑은** 에이전트들이 그 일을 대신합니다.

당신은 여전히 1인 개발자입니다. 하지만 당신은 더 이상 혼자가 아닙니다. 팀이 필요했고, 당신은 사람 없이 팀을 만들었습니다. 다섯 개의 사상이 그 일을 가능하게 했습니다.

세 중력을 인식했습니다. 프로토콜을 받아들였습니다. 하네스의 태도를 배웠습니다. 자기 채점의 함정을 피했습니다. 그리고 마지막으로 완벽이 아니라 **안 망하게**라는 철학을 받아들였습니다.

다섯 걸음이었습니다. 그리고 이 책은 여기서 끝납니다.

내일 아침 알람이 울리면, 당신이 어제 남겨둔 **progress.md**를 당신의 에이전트가 먼저 읽을 겁니다. 어제의 당신이 오늘의 당신에게 건넨 메모를요. 그 메모는 당신이 써서 당신이 읽는 것이 아닙니다. **당신이 써서 당신의 팀이 읽는 것**입니다.

당신의 다음 장은 당신이 쓸 차례입니다. 이 책 다음에 읽어야 할 책의 이름은 부록에 적어 두었습니다. 이 책이 **왜**에 답했다면, 그 책은 **어떻게**에 답합니다. 사상은 쉽고, 손은 느리게 따라옵니다. 그 손을 움직여 주는 책을 만나러 가십시오.

팀이 필요하다. 사람 말고.

이 책의 마지막 줄을 다시 한 번 적어 둡니다. 이 문장은 당신이 이 책을 펼치기 전부터 알고 있던 사실이고, 이 책을 덮은 지금은 증명된 약속입니다.

부록 · WHY의 다음은 HOW로

A. 이 다음에 읽을 책 — 『Claude Code로 1인 개발팀 만들기』

이 책은 왜에 답했습니다. 다섯 개의 사상 — 세 중력 · 프로토콜 · 하네스 · Gen/Val 분리 · 스케일의 철학 — 을 당신의 태도로 심었습니다.

어떻게는 다른 책에 있습니다. 같은 저자가 같은 시기에 펴낸 『Claude Code로 1인 개발팀 만들기』가 그 손을 움직이는 워크북입니다. 함께 읽기를 권장하지만 이 책 자체만으로도 읽힙니다. (자매 워크북은 AI 네이티브 개발의 사고방식을 넓게

다른 『AI 네이티브 개발 가이드』와도 이미 연결되어 있습니다. 본 책은 그 두 책 사이에 자리 잡은 **좁고 깊은 WHY**로 읽으시면 가장 잘 맞습니다.) 두 책의 관계를 한 장으로 정리하면 이렇습니다.

A-1. 두 책의 역할 분담

주제	(WHY · 사상)	『Claude Code로 1인 개발 팀 만들기』(HOW · 실전)
프로토콜	Ch2 — 왜 프롬프트가 아니라 프로토콜인가, Karpathy LLM Wiki의 의미	Ch3 — CLAUDE.md 4단계 진화, 필수 섹션, 글로벌·프로젝트 계층 구성
에이전트	Ch3 — 매뉴얼의 사상, SRP, Anthropic 2에이전트 구조	Ch4 — YAML frontmatter, 역할맵 4팀, 에이전트 .md 실제 예시
자동화	Ch3 — 하네스 5요소의 개념	Ch5 — 크론탭 · 비대화형 CLI · 실전 쉘 스크립트
품질 게이트	Ch3 — 위임 L1~L4 사상, 다층 방어원리	Ch6 — Hooks settings.json · pre-commit 혹은 배포 게이트
품질 게이트 시스템	Ch4 — 자기 채점의 함정 인식론, Gen/Val 분리 원리 (본 책 고유 프레임)	Ch6 — pre-commit · Hooks · Tier별 배포 게이트 실전
스케일	Ch5 — 완벽 vs 안 망하게, 티어링의 철학	Ch7·8·9 — 서비스 티어링 실전·실패 일지 7가지 사례 ·180일 로드맵

A-2. 독서 순서 권장

처음 Claude Code 1인 운영을 시작하는 분이면 이 순서를 권합니다.

1. 이 책(WHY)을 한 주 안에 가볍게 통독합니다. 사상만 흡수합니다.
2. 그 다음 『Claude Code로 1인 개발팀 만들기』(HOW)를 챕터별로 손을 움직이며 읽습니다.
3. 두 책의 같은 주제 챕터를 함께 펼쳐 두면 효과가 배가됩니다. 예: 이 책의 Ch2 + 워크북의 Ch3를 같은 날 읽기.

이미 실전 경험이 쌓여 있고 Level 4에서 정체된 분이면 순서가 다릅니다.

1. 이 책의 Ch3·Ch4만 먼저 집중해서 읽습니다. Anthropic 하네스·Gen/Val 상이 돌파구가 됩니다.
2. 워크북의 해당 장을 레시피만 참고해 구조를 보강합니다.
3. 이 책의 Ch5(스케일의 철학)는 서비스 3개 이상 운영하는 시점에 다시 열면 가장 큰 효과를 냅니다.

B. 참고 문헌 · 출처

B-1. Andrej Karpathy

- **Software 3.0**, YC AI Startup School 강연, 2025-06. Software 1.0 → 2.0 → 3.0 진화 프레임과 "LLM은 새로운 OS" 메타포의 출처. 이 책 Ch1의 지형도 배경.
- **LLM Wiki** 개념, 2026-04 공개 발언. 지식을 질의 시점이 아니라 ingestion 시점에 처리해 영속적·구조화된 위키형 산출물로 저장하는 아이디어. 이 책 Ch2의 핵심 앵커.
- **Context Engineering** 프레임, 2026. 선택·압축·순서·격리·포매팅 다섯 동사로 구성된 정보 설계 기술. 이 책 Ch3의 역할 분리·격리 사상의 근거.

B-2. Anthropic Engineering

- **Effective harnesses for long-running agents**, 2026. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents> 하네스 설계가 모델 성능보다 더 중요하다는 тезис.

Initializer + Coding 2에이전트 구조. structured artifacts로 세션 간 상태를 잇는 패턴. 이 책 Ch3의 중심 인용.

- Anthropic Engineering, GAN-style·Gen/Val 분리 관련 포스트 군.
<https://www.anthropic.com/engineering> 생성자와 평가자를 물리적으로 분리해야 주관 판단을 구체적 채점 가능 항목으로 변환할 수 있다는 원칙. 이 책 Ch4의 중심 근거.

B-3. 소프트웨어 원칙

- Robert C. Martin, **SOLID** 원칙 중 Single Responsibility Principle(SRP). 에이전트도 단일 책임이어야 한다는 사상의 이론적 근거. 이 책 Ch3.

B-4. 서사 프레임

- Donald Miller, **Building a StoryBrand**. 이 책의 제목, 프롤로그·에필로그 서사, 독자 주인공·악당 설정·변화 약속 구조의 배경. 세부는 본문에 노출하지 않았고 사상만 차용했습니다.

B-5. 온톨로지

- 온톨로지 철학 전통. 이 책은 "디렉토리로 도메인을 타이핑한다"는 **방식**만을 차용했습니다. 전통 자체는 다루지 않았습니다. 철학적 관심이 있으시면 별도 입문서 권장.

B-6. 가상 재구성

- 본 책의 모든 사례(Quiet Ark 스튜디오, PageMark·DeckBrew·Nightwatch 세 서비스, 새벽 3시 17분의 장애 등)는 저자의 1인 운영 경험을 **추상화·마스킹**하여 재구성한 가상 사례입니다. 실제 운영 서비스·실제 회사·실제 장애 로그의 직접 인용은 본 책에 포함되어 있지 않습니다.

C. 각 장 "내일 30분"의 한 장 요약

다섯 장을 읽는 동안 제시된 "내일 30분" 실천을 한 장으로 모아 둡니다. 이 페이지가 당신 책상에 붙어 있다면, 이 책의 사상이 **파일로 살기** 시작합니다.

- ☐ Ch1 — 자가진단 8문항 체크. 세 중력 중 가장 무거운 것 한 줄 메모.
- ☐ Ch2 — 자주 여는 프로젝트 하나에 빈 **CLAUDE.md** 생성. 기술 스택 5줄 + 반복 설명한 것 3줄만 씁니다.
- ☐ Ch3 — 같은 프로젝트에 **progress.md** 추가. 세 줄(하고 있는 것 / 막힌 것 / 다음 세션이 볼 것)만 쓰고, **CLAUDE.md** 맨 위에 "progress.md 먼저 읽기" 한 줄을 더합니다.
- ☐ Ch4 — 이번 주 짧은 글 또는 코드 변경 한 덩어리를 Gen → 새 세션 Critique 2단으로 돌려 봅니다. 차이 비교 30초.
- ☐ Ch5 — 서비스 목록에 Tier 1 / 2 / 3 표기. 가장 덜 중요한 서비스 하나의 모니터링 주기 한 단계 낮추기. 최근 장애 1건을 **FAILURES.md** 에 다섯 줄 기록.

다섯 줄의 실천이 쌓이면 당신의 새벽이 조용해집니다. 그 조용함이 이 책의 목적이었습니다.

저자 · Team Grit · 2026